

SVM and Naïve Bayes :

Theoretical :

1. What is a Support Vector Machine (SVM)?

A **Support Vector Machine (SVM)** is a supervised machine learning algorithm mainly used for classification and regression problems. In classification, SVM tries to find the best decision boundary, called a **hyperplane**, that separates data points belonging to different classes. The main objective of SVM is not simply to separate the classes but to find the hyperplane that provides the **maximum margin**, which means the maximum possible distance between the decision boundary and the closest data points from each class. The data points that are closest to the decision boundary are called **support vectors**, and they play an important role in determining the position of the boundary. For example, if we want to classify students as Pass or Fail using their study hours and attendance, SVM can find a line that separates the two groups while keeping the maximum distance from the closest students in both groups. For a linear SVM, the decision boundary can be represented as $w^T x + b = 0$, where w represents the weights and b represents the bias. SVM can also handle nonlinear classification problems by using **kernel functions** such as Linear, Polynomial, and RBF kernels. Therefore, SVM is a powerful algorithm that is especially useful for classification problems involving high-dimensional data and complex decision boundaries.

2. What is the difference between Hard Margin and Soft Margin SVM?

Hard Margin SVM and **Soft Margin SVM** differ mainly in whether the model allows training data points to violate the margin or be misclassified. In Hard Margin SVM, all training points must be correctly classified and no point is allowed to cross the margin. The constraint is $y_i(w^T x_i + b) \geq 1$. This approach works well when the data is clean and perfectly linearly separable, but it is highly sensitive to noise and outliers because even one incorrectly positioned point can make it impossible to find a suitable boundary. In contrast, **Soft Margin SVM** allows some data points to violate the margin or even be incorrectly classified. It introduces slack variables ξ_i and uses the constraint $y_i(w^T x_i + b) \geq 1 - \xi_i$. A penalty parameter C is used to control the cost of these violations. Therefore, Hard Margin SVM is strict and suitable for clean, perfectly separable data, whereas Soft Margin SVM is more flexible and is generally more suitable for real-world datasets containing noise and overlapping classes. In practical machine learning applications, Soft Margin SVM is more commonly used because real-world data is rarely perfectly separable.

3. What is the mathematical intuition behind SVM?

The mathematical intuition behind SVM is to find a **hyperplane that separates different classes while maximizing the margin** between them. For a linear SVM, the decision boundary is represented by $w^T x + b = 0$. The two boundaries defining the margin are $w^T x + b = 1$ and $w^T x + b = -1$. The distance between these two margin boundaries is given by $\frac{2}{\|w\|}$. Therefore, maximizing the margin is equivalent to

minimizing $\frac{1}{2} \|w\|^2$, subject to the condition $y_i(w^T x_i + b) \geq 1$. The points that lie closest to the decision boundary determine the margin and are called support vectors. A simple way to understand this is to imagine two groups of points on a graph and a ruler being placed between them. There may be many possible lines that separate the two groups, but SVM chooses the line that keeps the largest possible distance from the nearest points of both groups. This maximum-margin idea helps SVM achieve good generalization on unseen data.

4. What is the role of Lagrange Multipliers in SVM?

Lagrange Multipliers are mathematical variables used to solve the constrained optimization problem involved in SVM. SVM wants to minimize $\frac{1}{2} \|w\|^2$ while satisfying the classification constraint $y_i(w^T x_i + b) \geq 1$. Since this is a constrained optimization problem, Lagrange multipliers, represented by α_i , are introduced to combine the objective function and the constraints into a single mathematical expression called the **Lagrangian**. The Lagrangian for a basic SVM can be represented as $L(w, b, \alpha) = \frac{1}{2} \|w\|^2 - \sum_i \alpha_i [y_i(w^T x_i + b) - 1]$, where $\alpha_i \geq 0$. An important result of this process is that the training points having non-zero Lagrange multipliers become the **support vectors**. Thus, Lagrange multipliers help SVM solve its optimization problem, identify the important training points, formulate the dual optimization problem, and make it possible to use kernel functions. In simple terms, they help SVM determine which data points are important for constructing the best decision boundary.

5. What are Support Vectors in SVM?

Support Vectors are the training data points that are closest to the SVM decision boundary and therefore have the greatest influence on the position of the boundary. In a typical SVM classification problem, the decision boundary separates two classes and the support vectors are the points lying on or closest to the margin. These points are important because the position of the maximum-margin hyperplane is determined mainly by them. Data points that are very far from the decision boundary generally have little direct influence on the final boundary. For example, if an SVM is being used to separate two groups of students, the students whose values are closest to the boundary between Pass and Fail will become support vectors. If these support vectors move, the decision boundary may also change. Therefore, support vectors are a fundamental part

of SVM because they determine the margin and help the model make predictions for new observations.

6. What is a Support Vector Classifier (SVC)?

A **Support Vector Classifier (SVC)** is a classification algorithm based on the Support Vector Machine concept. It is used to classify data into two or more classes by finding a suitable decision boundary that separates the classes while attempting to maximize the margin. For binary classification, the decision function can be represented as $f(x) = w^T x + b$, and the predicted class depends on whether the value of this function is positive or negative. SVC can work with both linear and nonlinear data by using different kernel functions. Common kernels include the **Linear, Polynomial, RBF, and Sigmoid kernels**. In Python, SVC is available through the `sklearn.svm` module and can be implemented using `SVC(kernel="rbf")`. SVC is useful in applications such as image classification, text classification, medical diagnosis, and pattern recognition because it can produce effective decision boundaries even when the number of features is large.

7. What is a Support Vector Regressor (SVR)?

A **Support Vector Regressor (SVR)** is the regression version of Support Vector Machine and is used when the output variable is continuous rather than a class label. Instead of trying to separate different classes, SVR tries to find a function that predicts continuous values while keeping most of the training data within a specified error range called the **epsilon (ϵ) tube**. Errors that fall within this epsilon range are generally not penalized, while errors outside the range contribute to the loss. The basic idea can be represented as $|y - f(x)| \leq \epsilon$. For example, SVR can be used to predict house prices, temperature, electricity consumption, or other continuous values. SVR can use kernels such as Linear, Polynomial, and RBF, allowing it to model both linear and nonlinear relationships. In Python, it can be implemented using `from sklearn.svm import SVR` and `model = SVR(kernel="rbf")`. SVR is particularly useful when we want a regression model that focuses on controlling prediction errors while maintaining a suitable model complexity.

8. What is the Kernel Trick in SVM?

The **Kernel Trick** is a technique used in SVM to handle data that cannot be separated effectively using a straight-line or linear decision boundary. In some datasets, the relationship between the features and classes is nonlinear. Instead of explicitly transforming the original data into a higher-dimensional space, the kernel trick calculates the similarity between data points as if they had already been transformed into that higher-dimensional space. Mathematically, instead of directly calculating $\phi(x)^T \phi(z)$, SVM calculates this value using a kernel function $K(x, z)$. This allows SVM to construct nonlinear decision boundaries without explicitly creating the additional dimensions, which can save computational resources. Common kernel functions include

the **Linear Kernel, Polynomial Kernel, RBF Kernel, and Sigmoid Kernel**. For example, if two classes form circular or curved patterns that cannot be separated using a straight line, an RBF kernel can help SVM create a nonlinear boundary around the classes. Therefore, the kernel trick is one of the most important features of SVM because it allows it to solve complex nonlinear classification problems.

9. Compare Linear Kernel, Polynomial Kernel, and RBF Kernel.

The **Linear, Polynomial, and RBF kernels** are commonly used kernel functions in SVM, and they differ in the type of decision boundary they can produce. The **Linear Kernel** is the simplest kernel and is represented as $K(x, z) = x^T z$. It is suitable when the data is approximately linearly separable and is generally fast to train. The **Polynomial Kernel** is represented as $K(x, z) = (\gamma x^T z + r)^d$, where d represents the degree of the polynomial. It can model nonlinear relationships and is useful when the relationship between features has a polynomial structure. The **RBF or Radial Basis Function Kernel** is represented as $K(x, z) = e^{-\gamma \|x - z\|^2}$ and is capable of creating highly nonlinear decision boundaries. It is commonly used when the relationship between features is complex and there is no simple linear or polynomial boundary. Therefore, the Linear Kernel is simple and fast for linearly separable data, the Polynomial Kernel is useful for polynomial relationships, and the RBF Kernel is generally more flexible for complex nonlinear datasets.

10. What is the effect of the C parameter in SVM?

The **C parameter** in SVM controls the penalty given to classification errors and determines the balance between obtaining a wide margin and correctly classifying the training data. When a **small value of C** is used, the model gives less importance to individual classification errors and allows some points to violate the margin. This generally produces a wider and smoother margin and can help prevent overfitting, although an extremely small C can cause underfitting. On the other hand, when a **large value of C** is used, the model strongly penalizes classification errors and tries harder to correctly classify every training point. This can result in a narrower and more complex decision boundary and may increase the possibility of overfitting. For example, $SVC(C=0.1)$ allows more margin violations, while $SVC(C=100)$ places a much higher penalty on errors. Therefore, C is an important hyperparameter that should be selected carefully, usually through techniques such as cross-validation or GridSearchCV.

11. What is the role of the Gamma parameter in RBF Kernel SVM?

The **Gamma (γ) parameter** controls the influence or range of individual training points when an RBF kernel is used in SVM. The RBF kernel is represented by $K(x, z) = e^{-\gamma \|x - z\|^2}$. When gamma is **low**, each training point has a larger area of influence, resulting in a smoother and simpler decision boundary. If gamma is too low, the model may become too simple and underfit the data. When gamma is **high**, each training point

has a smaller area of influence, so the model creates a more detailed and complex decision boundary that closely follows the training data. If gamma is too high, the model may overfit. Therefore, a simple way to remember the effect is that **low gamma produces a smoother boundary, while high gamma produces a more complex boundary**. The appropriate gamma value depends on the dataset and is often selected using cross-validation.

12. What is the Naïve Bayes classifier, and why is it called "Naïve"?

Naïve Bayes is a supervised machine learning classification algorithm based on **Bayes' Theorem**. It predicts the class of a data point by calculating the probability of each possible class given the observed features and then selecting the class with the highest probability. It is called "**Naïve**" because it makes a simplified assumption that the features are **conditionally independent of one another given the class**. For example, when classifying an email as Spam or Not Spam, the model may consider words such as "free", "offer", and "money". Naïve Bayes assumes that these words are conditionally independent when the class is known, even though in reality words can be related to each other. For multiple features, the probability can be represented as $P(C, I, X_1, X_2 \dots X_n) \propto P(C) \prod_i P(X_i | C)$. Despite this strong assumption, Naïve Bayes is simple, fast, and often performs very well, especially in applications such as text classification, spam detection, and document classification.

13. What is Bayes' Theorem?

Bayes' Theorem is a mathematical rule used to calculate the probability of an event based on previously known information or evidence. It is the foundation of the Naïve Bayes classification algorithm. The formula is $P(A | B) = \frac{P(B|A)P(A)}{P(B)}$, where $P(A | B)$ is called the posterior probability, meaning the probability of event A after observing B; $P(B | A)$ is the likelihood, meaning the probability of observing B when A is true; $P(A)$ is the prior probability of A before observing the evidence; and $P(B)$ is the evidence or overall probability of B. For example, in spam detection, we may want to calculate the probability that an email is spam given that it contains the word "free". Bayes' Theorem allows us to combine our previous knowledge about spam emails with the probability of the word "free" occurring in spam emails. Naïve Bayes extends this idea by applying Bayes' Theorem to multiple features and using the conditional independence assumption.

14. Explain the differences between Gaussian Naïve Bayes, Multinomial Naïve Bayes, and Bernoulli Naïve Bayes.

Gaussian Naïve Bayes, Multinomial Naïve Bayes, and Bernoulli Naïve Bayes are different variants of the Naïve Bayes algorithm designed for different types of input data. **Gaussian Naïve Bayes** is used mainly for continuous numerical features and assumes that the features follow a Gaussian or normal distribution within each class. Examples

include age, height, weight, blood pressure, and temperature. **Multinomial Naïve Bayes** is commonly used for count-based data, particularly text classification, where features represent the number or frequency of words in a document. For example, a document may contain the word "free" five times and "offer" three times. **Bernoulli Naïve Bayes**, on the other hand, is designed for binary features where each feature generally represents whether something is present or absent, using values such as 0 and 1. For example, in text classification, a word may be represented as 1 if it is present in a document and 0 if it is absent. Therefore, Gaussian NB is suitable for continuous numerical data, Multinomial NB is suitable for count or frequency data, and Bernoulli NB is suitable for binary data.

15. When should you use Gaussian Naïve Bayes over other variants?

Gaussian Naïve Bayes should generally be used when the dataset contains **continuous numerical features** and these features can reasonably be modeled using a Gaussian or normal distribution within each class. Examples include healthcare datasets containing age, blood pressure, glucose level, BMI, and cholesterol values. In such situations, Gaussian NB estimates the mean and variance of each feature for each class and uses these values to calculate the probability of a new observation belonging to each class. It is preferred over Multinomial NB when the features are not word counts or frequencies and preferred over Bernoulli NB when the features are not binary 0/1 values. Gaussian NB is also useful when a simple and fast classification algorithm is required, particularly when the dataset is relatively small or medium-sized. However, if the input consists of word counts, Multinomial NB is generally more suitable, while binary presence/absence features are better suited to Bernoulli NB.

16. What are the key assumptions made by Naïve Bayes?

The most important assumption made by Naïve Bayes is that the input features are **conditionally independent given the class**. This means that once the class is known, the model assumes that the features do not depend on one another. For example, when classifying an email as Spam, Naïve Bayes may treat the presence of the words "free", "offer", and "money" as independent features after knowing that the email is Spam. Mathematically, the joint probability of several features can therefore be represented as the product of their individual conditional probabilities. Naïve Bayes also assumes that the probability distribution used by the selected variant is suitable for the data. For example, Gaussian NB assumes continuous features can be modeled using Gaussian distributions, while Multinomial NB is suitable for count-based features. Although the independence assumption is often not completely true in real-world datasets, Naïve Bayes can still perform effectively because the simplified probability calculations are computationally efficient.

17. What are the advantages and disadvantages of Naïve Bayes?

Naïve Bayes has several advantages that make it useful for classification problems. It is **simple and easy to implement**, requires relatively little training data, and is generally very **fast during both training and prediction**. It can handle high-dimensional datasets effectively and is particularly useful for text classification because text datasets often contain thousands of features. It can also provide probability estimates for different classes. However, Naïve Bayes also has some disadvantages. Its most important limitation is the assumption that features are conditionally independent, which may not be true in real-world datasets. Highly correlated features can therefore reduce the effectiveness of the model. Another problem is the **zero-frequency problem**, where a feature that has never appeared with a particular class can result in a probability of zero and cause the entire calculated probability to become zero. This problem can be handled using techniques such as **Laplace Smoothing**. The choice of Naïve Bayes variant is also important because different variants make different assumptions about the input data.

18. Why is Naïve Bayes a good choice for text classification?

Naïve Bayes is a good choice for **text classification** because text data usually contains a very large number of features, such as individual words, and Naïve Bayes can process such high-dimensional data efficiently. In applications such as spam detection, a document can be represented using words or word frequencies, and the model can calculate the probability that the document belongs to categories such as Spam or Not Spam. Naïve Bayes is especially effective because it works well with **sparse data**, where most possible words are absent from a particular document. It also has fast training and prediction times and does not require extremely large amounts of training data. **Multinomial Naïve Bayes** is particularly suitable when features represent word counts or frequencies, while **Bernoulli Naïve Bayes** can be used when features represent whether a word is present or absent. Because of its simplicity, speed, and effectiveness with high-dimensional sparse features, Naïve Bayes is widely used for spam detection, sentiment analysis, news classification, and document categorization.

19. Compare SVM and Naïve Bayes for classification tasks.

SVM and Naïve Bayes are both supervised classification algorithms, but they work using different approaches. SVM is primarily a **discriminative algorithm** that focuses on finding the best decision boundary between classes by maximizing the margin, whereas Naïve Bayes is a **probabilistic algorithm** that calculates the probability of each class using Bayes' Theorem. SVM generally does not require the features to be independent, while Naïve Bayes assumes that features are conditionally independent given the class. Naïve Bayes is usually faster to train and is particularly effective for high-dimensional sparse data such as text. SVM can also perform very well with high-dimensional data and has the additional advantage of using kernel functions to model nonlinear relationships. SVM generally requires more careful selection of parameters

such as C and gamma, while Naïve Bayes is comparatively simple to configure. Therefore, Naïve Bayes is often an excellent first choice for text classification and applications requiring speed and simplicity, while SVM can be preferred when finding a powerful decision boundary and handling complex classification patterns are more important.

20. How does Laplace Smoothing help in Naïve Bayes?

Laplace Smoothing is a technique used in Naïve Bayes to solve the **zero-frequency or zero-probability problem**. This problem occurs when a particular feature has never appeared in the training data for a particular class. Without smoothing, the probability of that feature given the class becomes zero. Since Naïve Bayes multiplies the probabilities of different features, even one zero probability can make the entire final probability zero. Laplace Smoothing solves this problem by adding a small value, usually 1, to every feature count before calculating probabilities. A commonly used formula is $\frac{\text{count}+1}{\text{total}+\text{number of possible values}}$. For example, if the word "offer" has never appeared in the training examples of a particular class, instead of assigning it a probability of zero, Laplace Smoothing gives it a small non-zero probability. This allows the model to continue calculating meaningful probabilities and prevents an unseen feature from completely eliminating a class. Therefore, Laplace Smoothing improves the robustness of Naïve Bayes, especially when working with text datasets where many words may not appear in every class.

Practical :

21. Write a Python program to train an SVM Classifier on the Iris dataset and evaluate accuracy.

This program uses the **Iris dataset**, which contains information about iris flowers and has three classes: Setosa, Versicolor, and Virginica. The data is divided into training and testing sets, and an SVM classifier is trained using the RBF kernel. The trained model predicts the classes of the test data, and the **accuracy score** is calculated by comparing the predicted values with the actual values. Feature scaling is used because SVM is sensitive to the scale of input features. The StandardScaler transforms the features so that they have approximately the same scale before they are given to the SVM model.

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score

# Load Iris dataset
iris = load_iris()

X = iris.data
y = iris.target

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Feature scaling
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Create SVM classifier
model = SVC(kernel="rbf", C=1)
```

```

# Train model
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)

# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)

print("Accuracy:", accuracy)

```

output : Accuracy: 0.97

22. Write a Python program to train two SVM classifiers with Linear and RBF kernels on the Wine dataset, then compare their accuracies.

This program uses the **Wine dataset** and trains two different SVM classifiers. The first classifier uses a **Linear kernel**, which creates a linear decision boundary, while the second uses an **RBF kernel**, which can create nonlinear decision boundaries. Since SVM is sensitive to feature scale, the features are standardized before training. Both models are trained using the same training and testing data, and their accuracies are calculated and compared. This helps us understand how different kernels can affect SVM performance.

```

from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score

# Load Wine dataset
wine = load_wine()

X = wine.data
y = wine.target

# Split data
X_train, X_test, y_train, y_test = train_test_split(

```

```

X, y, test_size=0.2, random_state=42
)

# Scale features
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Linear SVM
linear_model = SVC(kernel="linear")
linear_model.fit(X_train, y_train)

linear_pred = linear_model.predict(X_test)

linear_accuracy = accuracy_score(y_test, linear_pred)

# RBF SVM
rbf_model = SVC(kernel="rbf")
rbf_model.fit(X_train, y_train)

rbf_pred = rbf_model.predict(X_test)

rbf_accuracy = accuracy_score(y_test, rbf_pred)

# Display results
print("Linear Kernel Accuracy:", linear_accuracy)
print("RBF Kernel Accuracy:", rbf_accuracy)

```

23. Write a Python program to train an SVM Regressor (SVR) on a housing dataset and evaluate it using Mean Squared Error (MSE).

This program uses a housing dataset to predict house prices using **Support Vector Regression (SVR)**. Since SVR is sensitive to feature scale, the input features are standardized before training. The data is divided into training and testing sets, and an SVR model using the RBF kernel is trained on the training data. After prediction, the **Mean Squared Error (MSE)** is calculated. MSE measures the average squared difference between the actual and predicted values, so a lower MSE indicates better

regression performance. The example below uses the California Housing dataset available in Scikit-learn.

```
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVR
from sklearn.metrics import mean_squared_error

# Load housing dataset
housing = fetch_california_housing()

X = housing.data
y = housing.target

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Scale features
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Create SVR model
model = SVR(kernel="rbf")

# Train model
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)

# Calculate MSE
mse = mean_squared_error(y_test, y_pred)
```

```
print("Mean Squared Error:", mse)
```

24. Write a Python program to train an SVM Classifier with a Polynomial Kernel and visualize the decision boundary.

This program demonstrates how an SVM with a **Polynomial Kernel** can create a nonlinear decision boundary. To make visualization easy, only two features are used from a dataset. The model is trained using the polynomial kernel, and a mesh grid is created to display the regions predicted by the classifier. matplotlib is then used to plot the training points and the decision regions. This visualization helps us understand how the polynomial kernel separates different classes.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC

# Create two-dimensional dataset
X, y = make_moons(
    n_samples=200,
    noise=0.15,
    random_state=42
)

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Scale data
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Polynomial SVM
model = SVC(
```

```

    kernel="poly",
    degree=3,
    C=1
)

model.fit(X_train, y_train)

# Create mesh
x_min, x_max = X_train[:, 0].min() - 1, X_train[:, 0].max() + 1
y_min, y_max = X_train[:, 1].min() - 1, X_train[:, 1].max() + 1

xx, yy = np.meshgrid(
    np.linspace(x_min, x_max, 500),
    np.linspace(y_min, y_max, 500)
)

# Predict for every point in mesh
Z = model.predict(
    np.c_[xx.ravel(), yy.ravel()]
)

Z = Z.reshape(xx.shape)

# Plot decision boundary
plt.contourf(xx, yy, Z, alpha=0.3)

plt.scatter(
    X_train[:, 0],
    X_train[:, 1],
    c=y_train,
    edgecolors="k"
)

plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.title("SVM with Polynomial Kernel")
plt.show()

```

25. Write a Python program to train a Gaussian Naïve Bayes classifier on the Breast Cancer dataset and evaluate accuracy.

This program uses the Breast Cancer dataset provided by Scikit-learn and trains a Gaussian Naïve Bayes classifier. The dataset contains numerical features describing breast cancer cell characteristics and the target represents the diagnosis class. Since the features are continuous numerical values, Gaussian Naïve Bayes is suitable for this dataset. The dataset is divided into training and testing sets, the model is trained, predictions are generated, and accuracy is calculated using `accuracy_score`.

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score
```

```
# Load dataset
```

```
data = load_breast_cancer()
```

```
X = data.data
```

```
y = data.target
```

```
# Split data
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
# Create Gaussian Naive Bayes model
```

```
model = GaussianNB()
```

```
# Train
```

```
model.fit(X_train, y_train)
```

```
# Predict
```

```
y_pred = model.predict(X_test)
```

```
# Accuracy
```

```
accuracy = accuracy_score(y_test, y_pred)
```

```
print("Accuracy:", accuracy)
```

26. Write a Python program to train a Multinomial Naïve Bayes classifier for text classification using the 20 Newsgroups dataset.

This program demonstrates **Multinomial Naïve Bayes for text classification** using the 20 Newsgroups dataset. First, text documents are loaded from the dataset. The TfidfVectorizer converts the text into numerical TF-IDF features. Since MultinomialNB works with non-negative feature values, TF-IDF can be used because its values are non-negative. The classifier is then trained on the training documents and tested on unseen documents. Finally, the accuracy is calculated. Multinomial Naïve Bayes is commonly used for text classification because it can efficiently handle a large number of text features.

```
from sklearn.datasets import fetch_20newsgroups
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score
```

```
# Load training data
```

```
train_data = fetch_20newsgroups(
    subset="train",
    remove=("headers", "footers", "quotes")
)
```

```
# Load testing data
```

```
test_data = fetch_20newsgroups(
    subset="test",
    remove=("headers", "footers", "quotes")
)
```

```
# Convert text into numerical features
```

```
vectorizer = TfidfVectorizer(
    stop_words="english"
)
```

```
X_train = vectorizer.fit_transform(train_data.data)
```

```
X_test = vectorizer.transform(test_data.data)
```



```

y_train = train_data.target
y_test = test_data.target

# Create model
model = MultinomialNB()

# Train
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)

# Accuracy
accuracy = accuracy_score(y_test, y_pred)

print("Accuracy:", accuracy)

```

27. Write a Python program to train an SVM Classifier with different C values and compare the decision boundaries visually.

This program demonstrates how changing the **C parameter** affects the SVM decision boundary. A simple two-dimensional dataset is created so that the boundaries can be visualized easily. Three SVM models are trained using different C values. A smaller C allows more classification errors and generally produces a smoother boundary, while a larger C places more emphasis on correctly classifying training points and can produce a more complex boundary. The program plots the decision boundaries for comparison.

```

import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_moons
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler

# Create dataset
X, y = make_moons(
    n_samples=200,
    noise=0.2,
    random_state=42

```

```

)

# Scale features
scaler = StandardScaler()
X = scaler.fit_transform(X)

# Different C values
C_values = [0.1, 1, 100]

for C in C_values:

    model = SVC(
        kernel="rbf",
        C=C
    )

    model.fit(X, y)

# Create mesh
x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1

xx, yy = np.meshgrid(
    np.linspace(x_min, x_max, 400),
    np.linspace(y_min, y_max, 400)
)

Z = model.predict(
    np.c_[xx.ravel(), yy.ravel()]
)

Z = Z.reshape(xx.shape)

# Plot
plt.figure()

plt.contourf(xx, yy, Z, alpha=0.3)

```

```
plt.scatter(
    X[:, 0],
    X[:, 1],
    c=y,
    edgecolors="k"
)

plt.title("SVM Decision Boundary - C = " + str(C))
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")

plt.show()
```

28. Write a Python program to train a Bernoulli Naïve Bayes classifier for binary classification on a dataset with binary features.

This program demonstrates **Bernoulli Naïve Bayes**, which is designed for binary features containing values such as 0 and 1. A small example dataset is created where each feature can represent whether a particular condition is present or absent. The model is trained using `BernoulliNB`, predictions are generated for the test data, and the accuracy is calculated. Bernoulli Naïve Bayes is especially suitable when features represent the presence or absence of properties, such as whether a particular word occurs in a document.

```
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import BernoulliNB
from sklearn.metrics import accuracy_score
```

```
# Binary features
```

```
X = [
    [1, 0, 1, 0],
    [1, 1, 1, 0],
    [0, 0, 0, 1],
    [0, 1, 0, 1],
    [1, 0, 1, 1],
    [0, 1, 0, 0],
    [1, 1, 1, 1],
    [0, 0, 0, 0]
]
```

```

# Target classes
y = [1, 1, 0, 0, 1, 0, 1, 0]

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)

# Create Bernoulli Naive Bayes
model = BernoulliNB()

# Train
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)

# Accuracy
accuracy = accuracy_score(y_test, y_pred)

print("Accuracy:", accuracy)

```

29. Write a Python program to apply feature scaling before training an SVM model and compare results with unscaled data.

This program compares SVM performance **before and after feature scaling**. SVM calculates distances between data points, so features having very different scales can strongly affect the decision boundary. First, an SVM model is trained directly on the original data. Then StandardScaler is used to standardize the features, and another SVM model is trained using the scaled data. The accuracies of both models are displayed so that the effect of feature scaling can be observed.

```

from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score

# Load dataset

```

```

data = load_wine()

X = data.data
y = data.target

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# -----
# Without scaling
# -----

model_unscaled = SVC(kernel="rbf")

model_unscaled.fit(X_train, y_train)

pred_unscaled = model_unscaled.predict(X_test)

accuracy_unscaled = accuracy_score(
    y_test,
    pred_unscaled
)

# -----
# With scaling
# -----

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

model_scaled = SVC(kernel="rbf")

model_scaled.fit(
    X_train_scaled,

```

```

    y_train
)

pred_scaled = model_scaled.predict(X_test_scaled)

accuracy_scaled = accuracy_score(
    y_test,
    pred_scaled
)

# Display results
print("Accuracy without scaling:",
      accuracy_unscaled)

print("Accuracy with scaling:",
      accuracy_scaled)

```

30. Write a Python program to train a Gaussian Naïve Bayes model and compare the predictions before and after Laplace Smoothing.

This question requires an important clarification: **standard Gaussian Naïve Bayes does not normally use Laplace smoothing**, because Gaussian NB estimates a continuous probability distribution using the mean and variance of each feature rather than discrete feature counts. Laplace smoothing is mainly associated with discrete/count-based Naïve Bayes variants such as Multinomial and Bernoulli NB. Therefore, the program below demonstrates the concept by comparing Gaussian NB predictions with a manually smoothed version of the probability calculation. This is mainly for understanding the idea; in practical Scikit-learn work, Laplace smoothing should normally be demonstrated using MultinomialNB or BernoulliNB.

```

import numpy as np
from sklearn.naive_bayes import GaussianNB

# Simple continuous dataset
X_train = np.array([
    [1.0, 2.0],
    [1.2, 2.1],
    [4.0, 5.0],
    [4.2, 5.1]
])

```

```

y_train = np.array([0, 0, 1, 1])

X_test = np.array([
    [1.1, 2.0],
    [4.1, 5.0]
])

# Gaussian Naive Bayes
model = GaussianNB()

model.fit(X_train, y_train)

# Predictions before smoothing
pred_before = model.predict(X_test)

print("Predictions before smoothing:")
print(pred_before)

# Gaussian NB does not use Laplace smoothing.
# Laplace smoothing is mainly used with
# MultinomialNB and BernoulliNB.

print("\nNote:")
print("GaussianNB normally does not use Laplace smoothing.")
print("Use MultinomialNB or BernoulliNB for Laplace smoothing.")

# Demonstration using Bernoulli Naive Bayes
from sklearn.naive_bayes import BernoulliNB

X_binary = np.array([
    [1, 0],
    [1, 1],
    [0, 1],
    [0, 0]
])

y_binary = np.array([1, 1, 0, 0])

```

```

test_binary = np.array([
    [1, 1],
    [0, 1]
])

# alpha controls smoothing
model_smooth = BernoulliNB(alpha=1.0)

model_smooth.fit(X_binary, y_binary)

pred_after = model_smooth.predict(test_binary)

print("\nPredictions using Laplace smoothing:")
print(pred_after)

```

31. Write a Python program to train an SVM Classifier and use GridSearchCV to tune the hyperparameters (C, gamma, kernel).

This program uses **GridSearchCV** to automatically find the best combination of SVM hyperparameters. The parameters being tested are **C, gamma, and kernel**. Different values are supplied in a parameter grid, and GridSearchCV trains and evaluates the model using cross-validation for each combination. Finally, the best parameters and test accuracy are displayed. This approach is useful because selecting C, gamma, and kernel manually can be difficult.

```

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.pipeline import Pipeline
from sklearn.metrics import accuracy_score

# Load dataset
iris = load_iris()

X = iris.data
y = iris.target

# Split data

```



```

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Pipeline
pipeline = Pipeline([
    ("scaler", StandardScaler()),
    ("svm", SVC())
])

# Parameter grid
param_grid = {
    "svm__C": [0.1, 1, 10, 100],
    "svm__gamma": ["scale", 0.1, 1],
    "svm__kernel": ["linear", "rbf", "poly"]
}

# GridSearchCV
grid = GridSearchCV(
    pipeline,
    param_grid,
    cv=5,
    scoring="accuracy"
)

# Train
grid.fit(X_train, y_train)

# Predict
y_pred = grid.predict(X_test)

# Results
print("Best Parameters:")
print(grid.best_params_)

print("\nBest Cross-Validation Accuracy:")
print(grid.best_score_)

```

```
print("\nTest Accuracy:")
print(accuracy_score(y_test, y_pred))
```

32. Write a Python program to train an SVM Classifier on an imbalanced dataset and apply class weighting and check if it improves accuracy.

This program demonstrates how **class imbalance** can affect classification and how SVM can use the `class_weight` parameter to give greater importance to the minority class. An imbalanced dataset is created using `make_classification`, where one class contains many more samples than the other. First, an ordinary SVM is trained using the default class weights. Then another SVM is trained using `class_weight="balanced"`, which automatically assigns weights inversely proportional to class frequencies. The two models' accuracies are compared. However, accuracy alone is not always sufficient for imbalanced datasets, so precision, recall, and F1-score should also be considered.

```
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score
```

```
# Create imbalanced dataset
```

```
X, y = make_classification(
    n_samples=1000,
    n_features=10,
    n_classes=2,
    weights=[0.9, 0.1],
    random_state=42
)
```

```
# Split data
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42,
    stratify=y
)
```

```
# Scale features
```

```
scaler = StandardScaler()
```

```
X_train = scaler.fit_transform(X_train)
```

```

X_test = scaler.transform(X_test)

# SVM without class weighting
model1 = SVC(kernel="rbf")

model1.fit(X_train, y_train)

pred1 = model1.predict(X_test)

accuracy1 = accuracy_score(y_test, pred1)

# SVM with class weighting
model2 = SVC(
    kernel="rbf",
    class_weight="balanced"
)

model2.fit(X_train, y_train)

pred2 = model2.predict(X_test)

accuracy2 = accuracy_score(y_test, pred2)

print("Accuracy without class weighting:",
      accuracy1)

print("Accuracy with class weighting:",
      accuracy2)

```

33. Write a Python program to implement a Naïve Bayes classifier for spam detection using email data.

This program implements a simple **spam detection system** using Naïve Bayes. Email text is converted into numerical features using CountVectorizer, which represents each email using word counts. A **Multinomial Naïve Bayes** classifier is then trained using emails labeled as Spam or Not Spam. After training, new email messages are passed to the classifier, which predicts whether they are spam. This approach demonstrates why

Naïve Bayes is commonly used for email spam detection because it works efficiently with word-count features.

```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
```

```
# Email dataset
```

```
emails = [
    "Congratulations you won a free prize",
    "Claim your free money now",
    "Win a free lottery ticket",
    "Meeting is scheduled for tomorrow",
    "Please send the project report",
    "Your college assignment is due tomorrow",
    "Get free cash and win money",
    "Let's discuss the project today"
]
```

```
# 1 = Spam, 0 = Not Spam
```

```
labels = [
    1, 1, 1,
    0, 0, 0,
    1, 0
]
```

```
# Convert text to word-count features
```

```
vectorizer = CountVectorizer()
```

```
X = vectorizer.fit_transform(emails)
```

```
# Train Naive Bayes
```

```
model = MultinomialNB()
```

```
model.fit(X, labels)
```

```
# New emails
```

```
new_emails = [
    "You won free money",
    "Please attend the project meeting"
]
```

```

]

# Convert new emails
new_X = vectorizer.transform(new_emails)

# Predict
predictions = model.predict(new_X)

for email, prediction in zip(
    new_emails,
    predictions
):
    if prediction == 1:
        print(email, "-> Spam")
    else:
        print(email, "-> Not Spam")

```

34. Write a Python program to train an SVM Classifier and a Naïve Bayes Classifier on the same dataset and compare their accuracy.

This program trains two different classification algorithms, **SVM and Gaussian Naïve Bayes**, on the same Iris dataset. The data is divided into the same training and testing sets for both algorithms so that their performance can be compared fairly. Since SVM is sensitive to feature scale, the features are standardized before SVM training. Gaussian Naïve Bayes is then trained on the original numerical features. Both models make predictions on the same test data and their accuracy scores are compared.

```

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score

# Load Iris dataset
iris = load_iris()

X = iris.data
y = iris.target

```

```

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# -----
# SVM
# -----

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

svm_model = SVC(kernel="rbf")

svm_model.fit(
    X_train_scaled,
    y_train
)

svm_pred = svm_model.predict(X_test_scaled)

svm_accuracy = accuracy_score(
    y_test,
    svm_pred
)

# -----
# Naive Bayes
# -----

nb_model = GaussianNB()

nb_model.fit(X_train, y_train)

nb_pred = nb_model.predict(X_test)

```

```

nb_accuracy = accuracy_score(
    y_test,
    nb_pred
)

# Compare
print("SVM Accuracy:", svm_accuracy)
print("Naive Bayes Accuracy:", nb_accuracy)

```

35. Write a Python program to perform feature selection before training a Naïve Bayes classifier and compare results.

This program demonstrates how **feature selection** can be performed before training a Naïve Bayes classifier. The Iris dataset is used, and SelectKBest with the ANOVA F-test is used to select the most useful features. First, Gaussian Naïve Bayes is trained using all features. Then only the selected features are used to train another model. The accuracies of both models are compared. Feature selection can reduce unnecessary information, decrease model complexity, and sometimes improve generalization.

```

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.feature_selection import SelectKBest, f_classif
from sklearn.metrics import accuracy_score

```

```

# Load dataset
iris = load_iris()

```

```

X = iris.data
y = iris.target

```

```

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

```

```

# -----
# Model using all features
# -----

```

```

model_all = GaussianNB()

model_all.fit(X_train, y_train)

pred_all = model_all.predict(X_test)

accuracy_all = accuracy_score(
    y_test,
    pred_all
)

# -----
# Feature selection
# -----

selector = SelectKBest(
    score_func=f_classif,
    k=2
)

X_train_selected = selector.fit_transform(
    X_train,
    y_train
)

X_test_selected = selector.transform(
    X_test
)

# Train NB using selected features
model_selected = GaussianNB()

model_selected.fit(
    X_train_selected,
    y_train
)

pred_selected = model_selected.predict(

```



```

    X_test_selected
)

accuracy_selected = accuracy_score(
    y_test,
    pred_selected
)

# Results
print("Accuracy using all features:",
      accuracy_all)

print("Accuracy using selected features:",
      accuracy_selected)

print("Selected feature indices:",
      selector.get_support(indices=True))

```

36. Write a Python program to train an SVM Classifier using One-vs-Rest (OvR) and One-vs-One (OvO) strategies on the Wine dataset and compare their accuracy.

SVM is naturally designed for binary classification, but the Wine dataset contains three classes. Therefore, multiclass strategies are required. **One-vs-Rest (OvR)** creates one classifier for each class against all remaining classes, while **One-vs-One (OvO)** creates a classifier for every pair of classes. For three classes, OvR creates three classifiers, whereas OvO creates three pairwise classifiers. This program trains both approaches using the Wine dataset and compares their accuracies.

```

from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.multiclass import OneVsRestClassifier, OneVsOneClassifier
from sklearn.metrics import accuracy_score

# Load Wine dataset
wine = load_wine()

X = wine.data

```

```

y = wine.target

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

# Scale
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Base SVM
base_svm = SVC(kernel="rbf")

# One-vs-Rest
ovr_model = OneVsRestClassifier(base_svm)

ovr_model.fit(X_train, y_train)

ovr_pred = ovr_model.predict(X_test)

ovr_accuracy = accuracy_score(
    y_test,
    ovr_pred
)

# One-vs-One
ovo_model = OneVsOneClassifier(base_svm)

ovo_model.fit(X_train, y_train)

ovo_pred = ovo_model.predict(X_test)

```

```

ovo_accuracy = accuracy_score(
    y_test,
    ovo_pred
)

# Results
print("OvR Accuracy:", ovr_accuracy)
print("OvO Accuracy:", ovo_accuracy)

```

37. Write a Python program to train an SVM Classifier using Linear, Polynomial, and RBF kernels on the Breast Cancer dataset and compare their accuracy.

This program compares three commonly used SVM kernels: **Linear, Polynomial, and RBF**. The Breast Cancer dataset contains numerical features, and these features are standardized before training because SVM is sensitive to feature scale. Three separate SVM models are trained using the same training data and tested using the same test data. Their accuracy scores are calculated and displayed so that the performance of the three kernels can be compared.

```

from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score

```

```

# Load dataset
data = load_breast_cancer()

```

```

X = data.data
y = data.target

```

```

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

```

```

# Scale features

```

```

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

kernels = [
    "linear",
    "poly",
    "rbf"
]

for kernel in kernels:

    model = SVC(kernel=kernel)

    model.fit(X_train, y_train)

    y_pred = model.predict(X_test)

    accuracy = accuracy_score(
        y_test,
        y_pred
    )

    print(kernel, "Kernel Accuracy:", accuracy)

```

38. Write a Python program to train an SVM Classifier using Stratified K-Fold Cross-Validation and compute the average accuracy.

Stratified K-Fold Cross-Validation divides the dataset into several folds while maintaining approximately the same class proportions in every fold. This is particularly useful for classification problems because every fold contains a representative distribution of classes. In this program, the Iris dataset is divided into five stratified folds. The SVM model is trained and tested five times, with each fold being used once as the testing set. The individual accuracy scores and their average are then displayed.

```

from sklearn.datasets import load_iris
from sklearn.model_selection import StratifiedKFold, cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline

```

```
from sklearn.svm import SVC
```

```
# Load dataset  
iris = load_iris()
```

```
X = iris.data  
y = iris.target
```

```
# Create pipeline  
model = Pipeline([  
    ("scaler", StandardScaler()),  
    ("svm", SVC(kernel="rbf"))  
])
```

```
# Stratified K-Fold  
skf = StratifiedKFold(  
    n_splits=5,  
    shuffle=True,  
    random_state=42  
)
```

```
# Cross-validation  
scores = cross_val_score(  
    model,  
    X,  
    y,  
    cv=skf,  
    scoring="accuracy"  
)
```

```
print("Accuracy for each fold:")  
print(scores)
```

```
print("Average Accuracy:",  
      scores.mean())
```

39. Write a Python program to train a Naïve Bayes classifier using different prior probabilities and compare performance.

Prior probabilities represent the probability of each class before considering the input features. Naïve Bayes can either calculate class priors from the training data or use manually specified prior probabilities. This program uses Gaussian Naïve Bayes and compares a model using automatically learned priors with models using different manually specified prior probabilities. The models are evaluated using accuracy on the same test dataset. This demonstrates how changing prior beliefs can affect Naïve Bayes predictions.

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score
```

```
# Load dataset
iris = load_iris()
```

```
X = iris.data
y = iris.target
```

```
# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

```
# -----
# Default priors
# -----
```

```
model_default = GaussianNB()
```

```
model_default.fit(
    X_train,
    y_train
)
```

```
pred_default = model_default.predict(X_test)
```

```

accuracy_default = accuracy_score(
    y_test,
    pred_default
)

# -----
# Custom priors
# -----

model_custom = GaussianNB(
    priors=[0.6, 0.3, 0.1]
)

model_custom.fit(
    X_train,
    y_train
)

pred_custom = model_custom.predict(X_test)

accuracy_custom = accuracy_score(
    y_test,
    pred_custom
)

# Results
print("Default Prior Accuracy:",
      accuracy_default)

print("Custom Prior Accuracy:",
      accuracy_custom)

```

40. Write a Python program to perform Recursive Feature Elimination (RFE) before training an SVM Classifier and compare accuracy.

Recursive Feature Elimination (RFE) is a feature-selection technique that repeatedly trains a model and removes the least important features until the desired number of features remains. In this program, an SVM with a linear kernel is used as the estimator

because a linear SVM provides feature coefficients that RFE can use to determine feature importance. First, an SVM is trained using all features of the Breast Cancer dataset. Then RFE selects a smaller set of features, and another SVM is trained using only those selected features. Finally, the accuracies are compared.

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.feature_selection import RFE
from sklearn.metrics import accuracy_score
```

```
# Load dataset
```

```
data = load_breast_cancer()
```

```
X = data.data
```

```
y = data.target
```

```
# Split
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

```
# Scale
```

```
scaler = StandardScaler()
```

```
X_train = scaler.fit_transform(X_train)
```

```
X_test = scaler.transform(X_test)
```

```
# -----
```

```
# SVM using all features
```

```
# -----
```

```
model_all = SVC(kernel="linear")
```

```
model_all.fit(X_train, y_train)
```



```
pred_all = model_all.predict(X_test)
```

```
accuracy_all = accuracy_score(  
    y_test,  
    pred_all  
)
```

```
# -----  
# RFE  
# -----
```

```
estimator = SVC(kernel="linear")
```

```
rfe = RFE(  
    estimator=estimator,  
    n_features_to_select=10  
)
```

```
X_train_rfe = rfe.fit_transform(  
    X_train,  
    y_train  
)
```

```
X_test_rfe = rfe.transform(X_test)
```

```
# Train SVM  
model_rfe = SVC(kernel="linear")
```

```
model_rfe.fit(  
    X_train_rfe,  
    y_train  
)
```

```
pred_rfe = model_rfe.predict(  
    X_test_rfe  
)
```

```

accuracy_rfe = accuracy_score(
    y_test,
    pred_rfe
)

print("Accuracy using all features:",
      accuracy_all)

print("Accuracy using RFE features:",
      accuracy_rfe)

print("Number of selected features:",
      rfe.n_features_)

```

41. Write a Python program to train an SVM Classifier and evaluate its performance using Precision, Recall, and F1-Score instead of accuracy.

This program evaluates an SVM classifier using **Precision, Recall, and F1-score** rather than relying only on accuracy. Precision tells us how many observations predicted as positive were actually positive. Recall tells us how many actual positive observations were correctly identified. F1-score is the harmonic mean of precision and recall and provides a combined measure of both. These metrics are especially useful when the classes are imbalanced. The Breast Cancer dataset is used for this example.

```

from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import precision_score, recall_score, f1_score

# Load dataset
data = load_breast_cancer()

X = data.data
y = data.target

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,

```

```

    random_state=42,
    stratify=y
)

# Scaling
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# SVM
model = SVC(kernel="rbf")

model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)

# Metrics
precision = precision_score(
    y_test,
    y_pred
)

recall = recall_score(
    y_test,
    y_pred
)

f1 = f1_score(
    y_test,
    y_pred
)

print("Precision:", precision)
print("Recall:", recall)
print("F1-Score:", f1)

```

42. Write a Python program to train a Naïve Bayes Classifier and evaluate its performance using Log Loss (Cross-Entropy Loss).

Log Loss, also called **Cross-Entropy Loss**, evaluates the quality of predicted probabilities rather than simply checking whether the predicted class is correct. A lower Log Loss means that the model is assigning higher probabilities to the correct classes. Gaussian Naïve Bayes provides probability predictions using `predict_proba()`, which can be passed to `log_loss`. This program trains a Gaussian Naïve Bayes model on the Breast Cancer dataset and calculates its Log Loss.

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import log_loss
```

```
# Load dataset
```

```
data = load_breast_cancer()
```

```
X = data.data
```

```
y = data.target
```

```
# Split
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

```
# Gaussian Naive Bayes
```

```
model = GaussianNB()
```

```
model.fit(X_train, y_train)
```

```
# Probability predictions
```

```
y_prob = model.predict_proba(X_test)
```

```
# Log Loss
```

```
loss = log_loss(
    y_test,
```

```
        y_prob
    )

print("Log Loss:", loss)
```

43. Write a Python program to train an SVM Classifier and visualize the Confusion Matrix using seaborn.

This program trains an SVM classifier and evaluates it using a **Confusion Matrix**. A confusion matrix shows the number of correctly and incorrectly classified observations for each class. The matrix contains values such as True Positive, True Negative, False Positive, and False Negative for binary classification. seaborn is used to create a heatmap so that the matrix can be easily visualized. The Iris dataset is used in this example.

```
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import confusion_matrix

# Load dataset
iris = load_iris()

X = iris.data
y = iris.target

# Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

# Scale
scaler = StandardScaler()
```

```
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

```
# SVM
model = SVC(kernel="rbf")
```

```
model.fit(X_train, y_train)
```

```
# Predict
y_pred = model.predict(X_test)
```

```
# Confusion Matrix
cm = confusion_matrix(
    y_test,
    y_pred
)
```

```
# Visualize
plt.figure(figsize=(6, 5))
```

```
sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues"
)
```

```
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("SVM Confusion Matrix")
```

```
plt.show()
```

44. Write a Python program to train an SVM Regressor (SVR) and evaluate its performance using Mean Absolute Error (MAE) instead of MSE.

This program uses **Support Vector Regression (SVR)** to predict a continuous target value from a housing dataset. The California Housing dataset is used, and the input

features are standardized before training. After the SVR model makes predictions, **Mean Absolute Error (MAE)** is calculated. MAE represents the average absolute difference between actual and predicted values. Unlike MSE, MAE does not square the errors, making it easier to interpret because it is expressed in the same units as the target variable. A lower MAE indicates better performance.

```
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVR
from sklearn.metrics import mean_absolute_error
```

```
# Load housing dataset
data = fetch_california_housing()
```

```
X = data.data
y = data.target
```

```
# Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)
```

```
# Scale
scaler = StandardScaler()
```

```
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

```
# SVR
model = SVR(kernel="rbf")
```

```
model.fit(X_train, y_train)
```

```
# Predict
y_pred = model.predict(X_test)
```

```
# MAE
mae = mean_absolute_error(
    y_test,
    y_pred
)

print("Mean Absolute Error:", mae)
```

45. Write a Python program to train a Naïve Bayes classifier and evaluate its performance using the ROC-AUC score.

The **ROC-AUC score** measures how well a classification model can distinguish between different classes. ROC represents the relationship between the True Positive Rate and False Positive Rate at different classification thresholds, while AUC represents the area under this curve. An AUC value closer to 1 generally indicates better class separation, while a value around 0.5 indicates performance similar to random guessing. Since ROC-AUC requires probability or decision scores, `predict_proba()` from Gaussian Naïve Bayes is used. The Breast Cancer dataset is used for this example.

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import roc_auc_score
```

```
# Load dataset
data = load_breast_cancer()
```

```
X = data.data
y = data.target
```

```
# Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

```
# Gaussian Naive Bayes
model = GaussianNB()
```



```

model.fit(X_train, y_train)

# Probability for positive class
y_prob = model.predict_proba(X_test)[:, 1]

# ROC-AUC
auc = roc_auc_score(
    y_test,
    y_prob
)

print("ROC-AUC Score:", auc)

```

46. Write a Python program to train an SVM Classifier and visualize the Precision-Recall Curve.

A **Precision-Recall Curve** shows the relationship between precision and recall at different classification thresholds. It is particularly useful when the dataset is imbalanced because accuracy can sometimes give a misleading picture of performance. SVM can provide a decision score using `decision_function()`, which can then be used to calculate precision and recall at different thresholds. In this program, an SVM classifier is trained on the Breast Cancer dataset, the decision scores are calculated, and the Precision-Recall Curve is plotted using Matplotlib. The curve helps us understand the trade-off between identifying positive cases and avoiding false positive predictions.

```

import matplotlib.pyplot as plt

from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import precision_recall_curve

# Load dataset
data = load_breast_cancer()

X = data.data
y = data.target

```

```

# Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

# Scale features
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# SVM
model = SVC(kernel="rbf")

model.fit(X_train, y_train)

# Decision scores
y_scores = model.decision_function(X_test)

# Precision-Recall values
precision, recall, thresholds = precision_recall_curve(
    y_test,
    y_scores
)

# Plot curve
plt.figure(figsize=(7, 5))

plt.plot(
    recall,
    precision
)

plt.xlabel("Recall")
plt.ylabel("Precision")

```

```
plt.title("SVM Precision-Recall Curve")
```

```
plt.grid(True)
```

```
plt.show()
```